

概念鏈 skill:讓文檔的術語概念像變量一樣先定義後引用、可溯源

在 IDE 中讀代碼時遇到不認識的變量/函數，鼠標懸停即可顯示註釋說明，按一下 F12 就能轉到定義，看完後可再一鍵跳回原地。讀技術文檔時遇到不認識/不熟悉的術語，卻只能肉眼往前翻找定義；

```
28  
29     function yield_(this: void, candidate: Candidate): void  
30         把一個候選詞對象放進當前 translation 的輸出流  
31  
32         + @param candidate — 候選詞對象  
33     yield_(candidate)
```

圖 1 懸停顯示註釋



圖 2 vscode 右鍵菜單「轉到定義」

有沒有辦法能讓文檔像代碼一樣組織呢，遇到不認識不熟悉的術語就一鍵跳轉到定義，看完定義之後再跳轉回原處？

本 skill 就是專為解決這個問題而生。

1 | 使用例：講解減數分裂

接下來以減數分裂的講解為例，對比兩種寫法。先看通常的寫法。

1.1 | 原文

減數分裂（meiosis）是有性生殖生物在生殖細胞成熟過程中發生的特殊分裂方式，以精子的形成為例。減數分裂開始之前，精原細胞的每條染色體先複製一次；複製之後，每條染色體含有兩條姐妹染色單體，兩條由著絲粒連接在一起；

減數第一次分裂前期，同源染色體兩兩配對，這一現象叫聯會；聯會後，每對同源染色體含有四條染色單體，叫四分體；這一時期，非姐妹染色單體之間常發生交叉互換，相互交換一部分片段。

隨後，紡錘絲牽引聯會的同源染色體彼此分開，分別進入兩個子細胞，這是減數第一次分裂；此時著絲粒不分裂，染色體數目就此減半。

進入減數第二次分裂，兩個子細胞不再複製染色體，著絲粒一分為二，姐妹染色單體分開，成為兩條獨立的染色體，細胞再分裂一次，最終得到四個精細胞。減數分裂的結果是，成熟生殖細胞中的染色體數目比原始生殖細胞的減少一半。

你 tm 到底理解了需求和 skill 的作用沒有??? 甚麼東西都拿出來講

這份講解讀起來費勁。染色體、染色單體、著絲粒、同源染色體四個概念，全文只使用、不定義，讀者不認識時，編輯器幫不上忙，只能翻教材或查別處。聯會、四分體給了定義，但定義嵌在敘事裏，位置隨敘事而定，想回看只能肉眼往前翻；著絲粒在第三段「不分裂」、第四段「一分為二」處最要緊，全文搜「染色單體」，段段都有，沒有一句是定義。

不像人話

這些麻煩的源頭是同一個：概念的定義散落在正文各處，沒有固定的位置；正文提到概念，也只是普通文字，點不出定義在哪裏。概念鏈的寫法正好從這兩點下手。

1.2 | 說得不對怪

改寫

不對怪

同一份講解按概念鏈改寫，就是下面這份文檔：頂部引入框架，十條詞條組成登記區，正文在最後。

```
#import "_ConceptChain.typ":*

// — 概念鏈：登記順序 = 閱讀順序 —
// 強烈建議按照先聲明後引用的順序，不要前向引用搞亂順序
// 引用已登記的條目直接寫 @名字（用得最多）
// 引用後文才登記的條目寫 #R[@名字]（不建議）
// @名字後須跟空格或標點（@會吞緊隨的漢字）

#let Terms = [
  #MkR[染色體][
    染色體是細胞核內由 DNA 和蛋白質組成的結構，
    人有 23 對共 46 條。

    ]<染色體>

  #MkR[染色單體][
    染色單體是 @染色體 複製的產物：
    複製之後，
    每條 @染色體 含有兩條染色單體，
    兩條互稱姐妹染色單體，
    由 @着絲粒 連接在一起。

    ]<染色單體>

  #MkR[着絲粒][
```

着絲粒是 @染色體 上縊縮、
連接兩條姐妹 @染色單體 的部位。

]<着絲粒>

#MkR[同源染色體][

同源染色體是配對的兩條 @染色體：
形狀和大小一般都相同，
一條來自父方、一條來自母方。
人體細胞中的 23 對 @染色體，
每一對都是同源染色體。

（減數分裂中他們如何兩兩配對，
見 #R[@聯會]。）

]<同源染色體>

#MkR[聯會][

聯會指減數分裂過程中，
@同源染色體 兩兩配對的現象。

]<聯會>

#MkR[四分體][

四分體是 @聯會 後的結構：
由於每條 @染色體 都含有兩條姐妹 @染色單體，

聯會後的每對 @同源染色體 都含有四條 @染色單體，
所以叫作四分體。

]<四分體>

#MkR[交叉互換][

交叉互換指 @四分體 時期，
非姐妹 @染色單體 之間發生交叉、
相互交換一部分片段的現象。

]<交叉互換>

#MkR[減數第一次分裂][

減數第一次分裂中，
紡錘絲牽引聯會的 @同源染色體 彼此分開，
分別進入兩個子細胞；
此時 @着絲粒 不分裂，
姐妹 @染色單體 仍然相連，
染色體數目就此減半。

]<減數第一次分裂>

#MkR[減數第二次分裂][

減數第二次分裂緊隨 @減數第一次分裂，
基本過程與有絲分裂相似：
@着絲粒 一分爲二，

姐妹 @染色單體 分開，
成爲兩條獨立的 @染色體 ，
細胞隨之分裂，
產生四個精細胞。

]<減數第二次分裂>

#MkR[減數分裂][

減數分裂是進行有性生殖的生物，
在產生成熟生殖細胞時進行的染色體數目減半的細胞分裂：
減數分裂前 @染色體 複製一次，
細胞連續分裂兩次，
兩輪即 @減數第一次分裂 與 @減數第二次分裂 ；
成熟的生殖細胞 @染色體 數目比原始生殖細胞的減少一半。

]<減數分裂>

]

#Terms

// 以下是正文

以精子的形成爲例。
減數分裂開始之前，
精原細胞的每條 @染色體 先複製一次；
複製之後，

每條 @染色體 含有兩條姐妹 @染色單體，
兩條由 @着絲粒 連接在一起；

減數第一次分裂前期，
@同源染色體 兩兩配對，
這一現象叫 @聯會；
聯會後，
每對 @同源染色體 含有四條 @染色單體，
叫 @四分體；
這一時期，
非姐妹 @染色單體 之間常發生 @交叉互換，
相互交換一部分片段。

隨後，
紡錘絲牽引聯會的 @同源染色體 彼此分開，
分別進入兩個子細胞，
這是 @減數第一次分裂；
此時 @着絲粒 不分裂，
染色體數目就此減半。

進入 @減數第二次分裂，
兩個子細胞不再複製染色體，
@着絲粒 一分為二，

姐妹 @染色單體 分開，
成爲兩條獨立的 @染色體 ，
細胞再分裂一次，
最終得到四個精細胞。
@減數分裂 的結果是，
成熟生殖細胞中的 @染色體 數目比原始生殖細胞的減少一半。

這份文檔裏有四處值得停下來看。

畫引用關係圖

1. **鏈的壘法**。染色體是鏈頭，不引用任何人；著絲粒引用染色體；染色單體引用染色體與著絲粒；四分體引用聯會、同源染色體、染色單體；減數分裂引用兩輪分裂。讀者在正文裏 F12 「減數分裂」，跳到的詞條裏再跳「減數第一次分裂」，再跳「同源染色體」，一路到鏈頭爲止。每一跳都落在已經講過的概念上，不會踩空。還值得看減數分裂的位置：他是全文的總括概念，教科書習慣第一句就定義他，這條鏈卻把他排在鏈尾，因爲理

解他要先備齊前面全部零件。登記順序服務的是讀者的認識次序，不是名詞的大小。

2. **前向引用的實例**。兩處：染色體末尾的 `#R[@減數分裂]`，同源染色體末尾的 `#R[@聯會]`。兩處都是順手的延伸閱讀，不是理解當前詞條的前提，所以允許後置。
3. **詞條內的引用**。四分體的詳解裏「聯會後的每對 `@同源染色體` 都含有四條 `@染色單體`」就是詞條引用詞條：登記區自己也在用鏈，詞條不是幹癟的名詞卡，而是帶着論述的短文，彼此壘起來。這正是「鏈」這個說法的來歷。
4. **正文的引用**。正文裏十個概念的每次提及都是 `@` 引用，沒有一處裸文字。讀者從任何一段進入，遇到不認識的概念都是一次 F12 的事。

與原文對照，差別就在組織方式：每個概念的定義收攏在登記區的一條詞條裏，正文每處提及都是引用，F12 直達定義，返回導航跳回原處。

2 | 概念鏈寫法

這個 skill 約束的是 AI 講解時的輸出組織。你讓 AI 講解、解釋一樣東西，不論是一個流程、一套機制還是一門技術，AI 交回的都是一份按概念鏈寫法組織的 Typst 文檔，上一節的示例就是一份。

概念鏈寫法是：文檔頂部設一個概念登記區，示例中間那十條詞條就是登記區；講解要用到的每個關鍵概念先在登記區裏各佔一條詞條，寫足定義；正文提及概念處不寫裸文字，寫成指向詞條的引用。詞條的先後順序同時規定讀者認識概念的次序，概念一環扣一環，故名概念鏈。

引言裏那種 F12 直達定義、看完跳回原處的閱讀體驗，AI 的長篇講解特別需要。講解的每一段都是重新生成的，同一個概念前後換說法、隔幾段又解釋一遍，是散文式輸出的常見病；概念鏈把每個概念的定義收攏到唯一落點，把整篇講解釘在一致的詞彙上。

整套寫法基於 Typst 的標籤引用語法：元素後面跟 `<名字>` 就掛上標籤，`@名字` 就是引用，兩者都是 Typst 的原生能力。skill 不發明任何新語法，也就不需要安裝任何專門工具。

3 | 怎麼用

3.1 | 把講解任務交給 AI

掛上 skill 之後照常下指令：「講解減數分裂的流程」「解釋一下 C# 的純函數」，都算講解任務。AI 交回的 .typ 文檔，各項寫法規定在下一節的規約裏。

3.2 | 人怎麼讀

這套用法裏，.typ 源碼直接當文檔看。在 VSCode 裏裝上 Tinymist 插件打開源文件，正文就在眼前；遇到 @同源染色體 這樣的引用，F12 轉到定義，落點是該詞條的登記行，返回導航跳回原處接着讀。

渲染不是日常路徑：偶爾要把文檔發給別人看，typst compile 一句命令輸出 PDF，這是附帶能力。

4 | 寫法規約

AI 依照下面這些規約組織文檔； 人類作者想手寫概念鏈文檔， 照同一套規約辦即可。 規約分兩部分： 先看登記區的搭法， 再看四條順序與引用細則。

登記區的框架只有兩行：

```
// — 框架：文檔所有者定義，勿動 —  
#let MkR(display, descr) = figure(descr)  
#let R(cont) = {}
```

typst

MkR 是詞條工廠。 第一個參數是概念名， 第二個參數是詳解， 工廠把詳解排成一個 figure。 選 figure 沒有深意， 僅僅因為他是 Typst 裏現成的、能掛標籤被 @ 引用的元素， 隨手取用。 概念名參數不進渲染， 他在源碼裏充當詞條的標題行， 供人讀源碼時一眼認出這條是誰。

標籤寫在調用處。 詞條寫成 `#MkR[名字][詳解]<名字>`， 標籤緊跟在調用後面， 而不是在工廠內部用 `#label()` 集中構造。 好處直接落在 F12 上： 轉到

定義的落點是這一條詞條的登記行本身； 幾十個詞條各自落在各自的行， 而不會全部跳進工廠函數體內的同一行。

R 是前向引用標記。 他把傳入的內容直接丟棄， 不渲染， 編譯器也看不見裏面的名字。 他的用途只有一個： 把前向引用與普通引用在源碼裏區分開， 用法見第二條細則。

全部詞條收進一個內容變量， 放在文件頂部展開一次， 正文緊隨其後：

```
#let Terms = [  
  #MkR[染色體][  
    染色體（chromosome）是細胞核內由 DNA 與蛋白質組成的結構.....  
  ]<染色體>  
  
  #MkR[染色單體][  
    染色單體（chromatid）是 @染色體 複製的產物，  
    兩條拷貝由著絲粒相連.....  
  ]<染色單體>  
]  
  
#Terms
```

typst

框架與結構就這些；難的在順序與引用，下面四條細則管的就是這兩件事。

4.1 | 登記順序即閱讀順序

詞條的先後規定讀者認識這批概念的次序：基礎概念在前，組合概念在後，排序依據是「誰建立在誰之上」。例如講 C# 的純函數要動用三個概念：全局變量最基礎，排第一；副作用的常見情形之一是修改全局變量，建立在全局變量之上，排第二；純函數的兩個條件是確定性與無副作用，建立在副作用之上，排第三。

與順序配套的要求是：每條詞條只引用排在他前面的詞條。讀者從上往下走，點任何一個引用，跳到的詞條裏儘量不應再出現沒講過的東西；即使還有引用，指向的也是更靠前的、已經講過的概念。於是，不管讀者從哪個詞條進入，沿着 F12 一路走，路徑都是越走越基礎，最終到達鏈頭，全程不會踩空。

這條規約把詞條順序從美觀問題變成了邏輯問題：順序錯了，讀者就會在詞條裏撞見還沒登記的概念。

4.2 | 先聲明後引用，前向引用用標記

多數情況下，調整詞條順序就能做到先聲明後引用。實在繞不開、必須提及一個後文才登記的概念時，寫成 `#R[@名字]`，與普通引用 `@名字` 區分開。例如全局變量的詞條講完，想順手補一句「亂改全局變量惹出的麻煩，見後文的副作用」，而副作用排在後面，這一處就寫成 `#R[@副作用]`。

這個標記在源碼裏看得見，讀源碼時知道此處提及的是個後面的概念；渲染時他不出現。代價是編譯器看不見他：裏面的名字拼錯了不報錯，詳見驗證與邊界一節。所以規約明言：儘量避免前向引用，實在要用時才寫 `#R`。前面那個例子之所以被允許，是因為副作用在那裏只是順手的延伸閱讀，不是理解全局變量的前提；若他是前提，就必須調整詞條順序，把副作用提到全局變量前面。

4.3 | 新詞條插進鏈裏合適的位置

鏈是邏輯結構，新概念應當插進他邏輯上所屬的位置，而不是無腦挂在鏈尾。例如已有染色體、著絲粒、染色單體、同源染色體、四分體五條，

現在要登記「聯會」：四分體的講解要說自己是「聯會之後的結構」，聯會又建立在同源染色體之上，正確位置是插在同源染色體與四分體之間，而不是追加在四分體之後。

尾部追加省事，但每追加一條，基礎在前組合在後的次序就被破壞一分，登記區慢慢退化成雜亂的堆放場，登記順序即閱讀順序的承諾也就名存實亡。插入前多想一步，這條承諾才守得住。

5 | 維護與進階

5.1 | 已有文檔新增詞條：兩步流程

筆記寫到一半，正文出現了新概念，例如「姐妹染色單體」。把他納入概念鏈要分兩步走，順序不能顛倒。

第一步：在登記區裏插入詞條。先想清楚他建立在哪些已有概念之上，又有誰建立在他之上，然後插在邏輯合適的位置。姐妹染色單體是複製出的兩條拷貝的互稱，建立在染色單體之上；交叉互換講的是互換髮生在非姐妹染色單體之間，減數第二次分裂講的是姐妹染色單體分開，兩者的講解都依賴他，所以正確位置是緊插在染色單體之後，而不是追加在鏈尾。這一步依

循的正是「新詞條插進鏈裏合適的位置」那條規約：位置找準了，基礎在前組合在後的次序才守得住。

第二步：替換正文提及。 把正文中這個概念的裸文字提及全部換成引用寫法。這一步要看得仔細：同一個概念在正文裏可能換過好幾種說法，「姐妹染色單體」，甚至簡稱「兩條姐妹」，都可能是他。找出每一處，換準，不要漏。

漏換的後果不顯眼但實在：正文裏同時存在可跳轉的引用與不可跳轉的裸文字，讀者看不出兩者指的是同一個概念；日後概念改名時，引用處有編譯器報錯兜底，裸文字處則無人提醒，靜靜地爛掉。

替換時還會撞上 @ 的詞法問題。@ 會吞緊隨其後的漢字，所以引用後面必須跟空格或標點。有的句子直接換就能用：「姐妹染色單體，仍然相連」把開頭換成 @姐妹染色單體，後面正好接逗號，安全。有的句子就要順手調整措辭：「姐妹染色單體分開」若寫成 @姐妹染色單體分開，@ 後面緊跟漢字「分」，名字被吞長，編譯報錯。改寫前後對比：

改寫前：姐妹染色單體分開，各自成爲一條染色體。

改寫後：@姐妹染色單體 分開，各自成爲一條 @染色體 。

改寫後語序稍有變動，這是概念鏈對措辭的少量反向約束，成本不大，習慣即可。

5.2 | 概念表獨立與多文檔共用

一個項目的文檔往往不止一份：總覽、任務流程走讀、目錄樹導讀，各說各的事，但引用同一批概念。若每份文檔各自登記一遍詞條，同一個概念就有了好幾份定義，維護時要改好幾處，版本之間還會打架，等於讓使用例裏的問題捲土重來。

解法是把整條概念鏈抽成一個獨立文件，例如 `Concepts.typ`。框架函數與登記區都住在這個文件裏，各文檔用 `#include` 引入；標籤隨 `include` 落地，正文裏的 `@` 引用照常解析。概念從此只有一份定義，所有文檔共享。

引入方式有兩種，差別只在渲染：

- **可見落地。** 概念表文件末尾展開 `#Terms`，每份引入他的文檔渲染時都帶出整本登記簿。適合概念表本身就是正式交付物的場景。
- **隱形落地。** 寫成 `#hide[#include "Concepts.typ"]`。渲染時登記區不打印；但標籤照常注冊，全文的 `@` 引用一個不落地解析。適合某份文檔只想借用幾個概念、渲染時不想夾帶整本登記簿的場景。

在源碼直讀的模式下，這兩種寫法讀起來沒有分別：`#include` 進來的內容都在源碼裏，F12 都進得去，差別只有渲染時登記區出不出現。所以選哪種，取決於這份文檔將來要不要渲染給別人看、看的時候要不要附上詞彙表。

概念表獨立之後，還有一件事要想清楚：登記順序以誰為準。多份文檔的敘事次序未必相同，A 文檔裏自然的先後，到 B 文檔裏可能彆扭。共用一條鏈，就意味着順序只有一份，各文檔都要接受這份順序。多數項目的概念存在客觀分層，基礎設施在下、業務組合在上，以這個分層定順序通常都能令各方滿意；實在衝突時，寧可回歸各自登記，也不要讓鏈爲了遷就某一份文檔而犧牲邏輯次序。

5.3 | 驗證與邊界

驗證手段只有一條命令：`typst compile`。一次編譯就是一次全量檢查：正文引用了未登記的名字，報錯；名字拼錯了一個字，報錯；`@`吞了後面的漢字導致名字變長，同樣報錯。寫文檔寫出了編譯期檢查：引用與詞條對不上，編譯就通不過。這正是從代碼那裏一併搬來的性質。

唯一的盲區是前向引用標記 `R` 的內部。內容被丟棄，編譯器看不見裏面的名字，拼錯不報錯。自查的辦法笨但可靠：全文搜索 `#R[`，逐個人工核對。這也是規約不建議前向引用的原因：少用，盲區就小。

適用邊界也該說清。概念鏈適合三類場景：

- 概念密度高的長文檔：教程、規範、源碼走讀屬於此類。概念反覆出現，登記一次攤薄全部成本。
- 多人或多個 AI 協作維護的文檔：提及即引用的紀律加上編譯檢查，讓後來者不敢裸寫，漏寫瞞不過編譯。
- 一套概念支撐多篇文檔的項目：概念表獨立後天然共享，上一節整節講的就是這件事。

不值得用的場景也明確：兩三個概念的短文，框架代碼比正文還長，登記的儀式成本蓋過收益，不如就地解釋了事。

6 | 來歷與設計哲學

示例、用法、規約與維護到此講完。最後一節回過頭講這套寫法的來歷：他從讀代碼的體驗出發，經過一次用編程語言直接寫文檔的嘗試，最終落在 Typst 的標籤引用語法上。不了解來歷不影響使用；但框架裏幾處取舍的所以然，放在這裏才講得清楚。

6.1 | 讀代碼是怎麼讀的

沒有人從第一行順讀到最後一行地去讀一個陌生代碼庫。讀代碼是跳着讀的：引言裏那套懸停看註釋、F12 轉到定義、再跳回原處的操作，就是讀代碼的日常；整個過程閱讀不中斷，注意力一直留在當前的問題上。

這套體驗能成立，靠的是代碼的兩個性質。其一，定義有唯一落點：每個函數只定義一次，轉到定義永遠落在同一處。其二，提及處就是引用：每個調用處在語法上就是一個指向定義的引用，編輯器認得他，所以能一鍵直

達。在這個模式下，讀者不需要事先認識所有函數，遇到不認識的，跳過去看一眼就是了。

6.2 | 讀文檔沒有這個待遇

散文式的文檔享受不到這些。使用例一節已經看到：同一個概念解釋了一遍又一遍，讀者想確認術語只能中斷閱讀往回翻。這些問題的根源，在於散文式文檔天生缺了代碼的那兩個性質：定義沒有唯一落點，提及處不是引用。

6.3 | 靈感：把文檔寫成代碼

於是想到一個直接的辦法：既然代碼有定義和引用，那就把文檔寫成代碼。寫一個工廠函數 `createTerm`，概念經由他登記成變量：名字與詳解一起傳入；正文寫成一個長模板字符串，提及概念處用插值引用變量。落到 TypeScript 上，這份減數分裂筆記長這樣：

```
function createTerm(display: string, descr: string) {...}

// — 登記區 —
```

ts

```
let 染色體 = createTerm("染色體", "細胞核內由 DNA 與蛋白質組成的結構，遺傳物質的主要載體.....（詳解）")
```

```
let 染色單體 = createTerm("染色單體", `一條 ${染色體} 複製出的兩條拷貝之一，兩條由著絲粒相連.....（詳解）`)
```

```
let 同源染色體 = createTerm("同源染色體", `配對的兩條 ${染色體}，一條來自父方、一條來自母方.....（詳解）`)
```

```
let 四分體 = createTerm("四分體", `配對的一對 ${同源染色體}，共含四條 ${染色單體}.....（詳解）`)
```

正文也是代碼，寫成一個長模板字符串，提及處用插值：

```
// — 以下是正文 —
```

```
let 正文 = `
```

```
跟蹤一個精原細胞的旅程：
```

```
先複製 ${染色體}，
```

```
再讓 ${同源染色體} 配對分離.....
```

```
`
```

ts

在 IDE 裏打開這個 .ts 文件，讀代碼的整套體驗立刻免費生效：在正文裏看到 `${同源染色體}`，不認識就 F12，光標跳到

`let 同源染色體 = createTerm(...)` 那一行，看完返回導航跳回來接着讀。定義有唯一落點，提及處是語法上的引用，兩個性質都齊了，使用例裏的問題似乎就此清零。

6.4 | 代碼當文檔，不方便

但真拿編程語言寫文檔，有兩個不方便很快浮上來。

其一，不能前向引用。多數語言要求先聲明後使用。講解一個概念時，你常常想順手提一句後面的概念：染色體的詞條講完，想補一句「這 46 條之後會減半傳給下一代」，於是寫下：

```
let 染色體 = createTerm("染色體", `細胞核內由 DNA 與蛋白質組成的結構..... (詳解)
他在 ${減數分裂} 時減半傳給下一代`)
let 減數分裂 = createTerm("減數分裂", `複製一次、連續分裂兩次..... (詳解)`)
```

第一行的初始化引用了還沒宣告的 `減數分裂`，編譯器直接報錯，理由是在宣告之前使用了這個變量。這不是誤用，是語言的硬約束：`let` 的初始化

立即求值，右邊不許出現任何後面的名字。想提後面的概念，只能要麼硬調整詞條順序遷就這一句話，要麼忍着不提。可概念的講解順序應該由邏輯決定，不該被語言的求值規則綁架。

其二，寫起來麻煩。 文檔的主體是散文，用字符串裝散文，段落靠手拼換行，列表要自己造格式，標點排版一概沒有，代碼高亮也別想。書寫體驗是在寫代碼，閱讀體驗也是在讀代碼：你得到了導航，失去了文檔本來的形態。

6.5 | Typst：現成的落點

換個思路：與其勉強用編程語言充當文檔語言，不如找一種本來就是代碼的文檔語言。Typst 正是這樣的落點。`.typ` 源碼本身既是文檔又是代碼：他有原生的標籤機制，任何元素後面跟一個 `<名字>` 就掛上了標籤；`@名字` 就是引用。於是把文檔寫成 Typst，讀代碼的體驗整套搬進來，而且不需要任何發明，用的全是語言的原生能力。

最關鍵的一步對比在前向引用。Typst 的標籤不分先後，全文生效：`@` 引用一個掛在文件末尾的標籤，照樣解析，照樣可跳。前向引用在語言層面完全合法，TypeScript 裏那個報錯在這裏不存在。於是「先聲明後使用」從

語言的硬約束降格為編輯上的規約：順序不再由編譯器把關，得由作者自己守。怎麼守，就是前文「寫法規約」一節的四條細則。

7 | 小結

概念鏈 skill 的主張一句話可以講完：讓 AI 講解東西時，把講解組織成一份概念先登記、提及即引用的 Typst 文檔，使讀者得到與讀代碼同款的導航體驗：定義有唯一落點，提及處就是引用。

支撐這個主張的東西一共四樣：Typst 的標籤引用語法提供基座；兩個框架函數搭起登記區；四條規約守住概念次序；一條 `typst compile` 兜底驗證。來歷一節交代了選型：編程語言試過了，導航免費，但先聲明後使用是硬約束，散文也塞不進字符串；Typst 接住了，前向引用在語言層合法，順序降格為編輯規約。